UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2022/2023

MAY EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

WEDNESDAY, 10 MAY 2023 TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions .

Answer **ALL** questions in **Section A** and **ONLY ONE(1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the FIRST question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

SECTION A (Answer all questions)

Q1. You want to build a model to predict if a customer will buy a product (label = 1) or not (label = 0). You collected a total of 1000 examples with the following features: age (10 to 80 years old), income (500 to 100000), and gender (male or female).

(a) As a baseline model, you decide to train a logistic regression.

(i) If the shape of $x^{(i)}$ is $(n_x, 1)$, what should n_x be? (2 marks)

(ii) How many parameters do the model have? (2 marks)

(iii) Why is it good to standardize the input features for this task? (4 marks)

(b) You decide to train a standard neural network with a single hidden layer with 80 hidden neurons to implement the model.

(i) Why use a neural network with multiple layers? (3 marks)

(ii) How many parameters do this neural network have? (3 marks)

(iii) Explain why it is a bad idea to use linear activation for the hidden layer. (3 marks)

(iv) You want to enrich the input features with the history of the customer, propose specifically the features that you will add to the feature vector. (5 marks)

(v) Specify the necessary changes to the system above so that you can train a network to predict if a customer will purchase any of the N products sold in the shop. A customer may purchase more than one product. (3 marks)

[Total : 25 marks]

Q2. (a) (i) Explain why Convolutional Neural Network (CNN) is preferred over Standard Neural Network for processing image data. (2 marks)

(ii) Given the following input activation map of shape (1, 5, 5) and a 3x3 filter for a convolutional layer (padding = 0, stride = 2), compute the output activation map.

Input activation map

-16	3	0	2	8
4	-24	2	4	-2
-8	4	28	3	2
16	32	-4	2	-12
4	2	16	20	-24

Filter

1	½	0
0	0	0
2	¼	1

(8 marks)

(iii) What is the shape of the output activation map if we feed an input of shape (B, C, H, W) = (4, 256, 118, 118) to a convolutional channel with the following settings: kernel size = 3, stride = 3, padding = 1, number of filters = 128? (3 marks)

(b) Gradient Descent (GD) is used to find the minimum of the following function:

$$f(x) = 3x^2 - 2x + 2$$

(i) Does Gradient Descent always find the minimum of $f(x)$? Justify your answer. (3 marks)

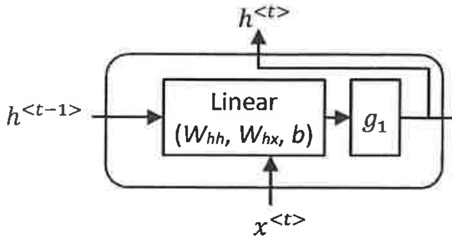
(ii) Initialize $x = 0$ and set learning rate $lr = 0.1$. Perform gradient descent for the first 3 steps. Round your answer to 2 decimal points in each step.

t	x	$f(x)$	dx
0			
1			
2			

(9 marks)

[Total : 25 marks]

Q3. (a) Figure 3.1 shows a simple RNN cell and the forward propagation formulae.



$$h^{<t>} = g_1(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b)$$

Figure 3.1: Simple RNN cell

(i) Where do the signals $h^{<t-1>}$ and $x^{<t>}$ in an RNN cell come from in an unrolled RNN network? (2 marks)

(ii) Explain why the RNN cell have problem with vanishing gradient effect. (2 marks)

(iii) Given length of the input vector $x^{<t>}$ (i.e., the input dimension) = 100 and the number of units (i.e., the hidden dimension) = 32. What is the shape of network parameters W_{hh}, W_{hx} and b? (3 marks)

(b) Consider a batch data for sentiment analysis (0: negative, 1: positive) on user comment. There are 1000 unique words (including special tokens) in the dataset.

Table 3.1: Batch data (batch size = 2) for sentiment analysis

Input	Sentiment
What a wonderful movie	1
A waste of time	0

(i) Explain why a RNN model is selected for the task. (2 marks)

(ii) Draw the unrolled version of a *single-layered uni-directional RNN* for the batch data in Table 3.1. Attach a *fully connected* layer to the RNN output to generate the output of the network. (8 marks)

(iii) Must the length of the input sequence be the same for all samples in a batch? What about samples from different batches? Justify your answer. (4 marks)

(iv) Suppose that one-hot encoding is used to train the network on the batch data in Table 3.1.

- What is the size of the input feature for a single sample at time t?
- Suppose the batch first format is used. What is the shape of the input matrix X for the batch data?

(4 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

SECTION B (Choose ONE question only)

Q4. (a) Given a large unlabeled image dataset, SimCLR can be used to learn a pre-trained model, which can then be finetuned for a variety of downstream tasks.

(i) Draw the network structure for SimCLR to pre-train the model with a pretext task. Use the diagram to explain the pretext task. (10 marks)

(ii) Explain how the network structure in (a)(i) can be adapted for a downstream task. (5 marks)

(iii) For data augmentation, we randomly sampled noise value a and add it to all pixels in the image X to get $X + a$. Justify if the applied data augmentation is appropriate for SimCLR. (2 marks)

(b) A real-time face recognition system is implemented in Caffe-ResNet-101. It barely meets the maximum memory and minimum accuracy requirement. Its inference speed is around half of what is needed for a real-time system.

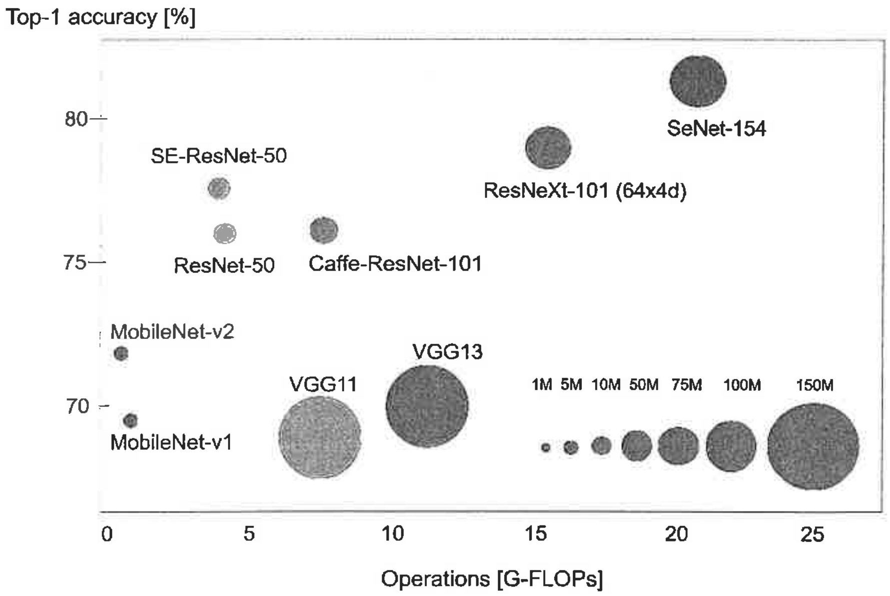


Figure 4.1: Comparison of CNN Architectures

(i) Refer to Figure 4.1. Is it reasonable to replace Caffe-ResNet-101 with the following networks? Provide specific justification for your answer.

- SENet-154
- MobileNet-v2
- SE-ResNet-50

(6 marks)

(ii) VGG networks have a much higher footprint compared to other network architectures. Why? (2 marks)

[Total : 25 marks]

Q5. (a) The task is to implement a language model with the network. Teacher forcing is used to train the model.

(i) We can create a training set from some raw text using the shifted portion of sentence as labels. Given the following raw text: "*ChatGPT is a powerful language model*", construct the training set by filling up the table below. The length of the input and output sequences are set to 3.

Input	Expected output

(6 marks)

(ii) When performing teacher forcing during *training* phase, justify if sampling is needed during training. (3 marks)

(iii) Why is sampling needed during *evaluation* (testing) phase? (3 marks)

(iv) An RNN network can be used to implement the language model. Draw on top of the RNN network in Figure 5.1 to implement a language model and generate novel text during *evaluation* phase. Tips: Remember to add the sampling sub-blocks into the diagram.

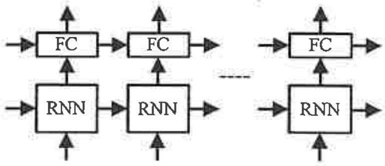
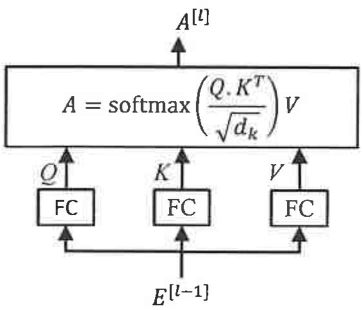


Figure 5.1 RNN network (6 marks)

(b) The following a self-attention module in a Transformer layer



(i) Justify why the transformer layer is more efficient than RNN layer. (3 marks)

(ii) What are the full names for Q, K and V? (2 marks)

(iii) Elaborate what does $Q.K^T$ represent? (2 marks)

[Total : 25 marks]

Appendix: Formulas

Logistic Regression:

$z = w_1x_1 + \cdots + w_{n_x}x_{n_x} + b$ (non-vectorized)

$z = w^Tx + b$ (vectorized)

$\sigma(z) = \frac{1}{1+e^{-z}}$

$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}\left(-y^{(i)}\ln\hat{y}^{(i)} - \left(1-y^{(i)}\right)\ln\left(1-\hat{y}^{(i)}\right)\right)$

$w := w - \alpha\frac{\partial J(w,b)}{\partial w}$

$b := b - \alpha\frac{\partial J(w,b)}{\partial b}$

Backpropagation for Logistic Regression:

(Non-vectorized)

$dz^{(i)} = \hat{y}^{(i)} - y^{(i)}$ activation phase

$dw_j^{(i)} = x_j^{(i)}.dz^{(i)}$ summation phase

$db^{(i)} = dz^{(i)}$ summation phase

(Vectorized)

$dZ = \hat{Y} - Y$ activation phase

$dw = \frac{1}{m}(X.dZ^T)$ summation phase

$db = \text{np.mean}(dZ)$ summation phase

Standard Neural Network:

$Z^{[i]} = W^{[i]}A^{[i-1]} + b^{[i]}$ summation phase for $i = 1 \dots L$

$A^{[i]} = g\left(Z^{[i]}\right)$ activation phase for $i = 1 \dots L$

$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{C}e^{z_j}}$ softmax function

$J(\hat{Y},Y) = \frac{1}{m}\sum_{i=1}^{m} -\ln\left(\hat{y}_{y^{(i)}}^{(i)}\right)$ cross entropy function where $\hat{y}_{y^{(i)}}^{(i)}$ is the probit score $\hat{y}^{(i)}$ for the true class $y^{(i)}$

(Activation function)

$g(z) = \frac{1}{1+e^{-z}}$ sigmoid activation

$g'(z) = g(z)\left(1-g(z)\right)$

$g(z) = \frac{e^z-e^{-z}}{e^z+e^{-z}}$ tanh activation

$g'(z) = 1 - g(z)^2$

$g(z) = \max(0,z)$ relu activation

$$g'(z) = \begin{cases} 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \\ 0 & \text{if } x < 0 \end{cases}$$

$g(z) = \max(0.01z, z)$ leaky relu activation

$$gg'(z) = \begin{cases} 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \\ 0.01 & \text{if } x < 0 \end{cases}$$

$g(z) = \max(\alpha z, z)$ parametric relu

$$g'^{(z)} = \begin{cases} 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \\ \alpha & \text{if } x < 0 \end{cases}$$

$$g(z) = \begin{cases} z & \text{if } x > 0 \\ \alpha(e^z - 1) & \text{if } x \le 0 \end{cases}$$ Exponential relu

$g(z) = \max(w_1^Tz + w_2^Tz + b)$ Parametric relu

Backpropagation for the Standard Neural Network:

$dZ^{[i]} = dA^{[i]} * g^{[i]\prime}(Z^{[i]})$ Activation phase

$g^{[i]\prime}\left(Z^{[i]}\right) = A^{[i]} * \left(1 - A^{[i]}\right)$ Activation phase (for sigmoid activation)

$g^{[i]\prime}\left(Z^{[i]}\right) = Z^{[i]} > 0$ Activation phase (for relu activation)

$dW^{[i]} = \frac{1}{m}dZ^{[i]}A^{[i-1]^T}$ Summation phase

$db^{[i]} = \text{np.mean}\left(dZ^{[i]}\right)$ Summation phase

$dA^{[i-1]} = W^{[i]^T}dZ^{[i]}$ Summation phase

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Convolutional Neural Network:

size $= \left\lfloor \frac{i+2p-f}{s} + 1 \right\rfloor$	size of the activation map
#params in a filter $= c_f \times h_f \times w_f + 1$	Number of parameters in a filter
#params in a conv layer = #filters × #params in a filter	Number of parameters in a CONV layer
#computations = #activations in output × #params in a filter = size of output activation map × #params in a filter	Number of computations in a CONV layer
depth $d = \alpha^{\phi}$, width $w = \beta^{\phi}$, resolution $r = \gamma^{\phi}$ subject to: $\alpha.\beta^2.\gamma^2 \approx 2$ $\alpha \geq 1, \beta \geq 1, \gamma \geq 1$	Efficient Net formulation

RNN:

$h^{<t>} = g_1(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b)$	Simple RNN Cell
$h^{<t>} = g_1\left(W_h \begin{bmatrix} h^{<t-1>} \\ x^{<t>} \end{bmatrix} + b\right)$	Simple RNN Cell (Simplified form)

LSTM:

$f^{<t>} = \sigma(W_f(h^{<t-1>}, x^{<t>}) + b_f)$	forget gate
$i^{<t>} = \sigma(W_i(h^{<t-1>}, x^{<t>}) + b_i)$	input gate
$o^{<t>} = \sigma(W_o(h^{<t-1>}, x^{<t>}) + b_o)$	output gate
$\tilde{c}^{<t>} = \tanh(W_c(h^{<t-1>}, x^{<t>}) + b_c)$	new content (raw)
$c^{<t>} = i^{<t>} * \tilde{c}^{<t>} + f^{<t>} * c^{<t-1>}$	cell state
$h^{<t>} = o^{<t>} * \tanh(c^{<t>})$	hidden state

Attention-based RNN:

$h_d^{<t_d>} = g(W_{hh}h^{<t_d-1>} + W_{hx}x^{<t_d>} + W_{hc}c^{<t_d>} + b)$	output hidden state of a decoder RNN cell
$e^{<t_d,t_e>} = \text{fc}(h_e^{<t_e>}, h_d^{<t_d-1>})$, $\alpha^{<t_d,t_e>} = \frac{\exp(e^{<t_d,t_e>})}{\sum_{t_e=1}^{T_e}\exp(e^{<t_d,t_e>})}$, $c^{<t_d>} = \sum_{t_e=1}^{T_x} \alpha^{<t_d,t_e>} h^{<t_e>}$	formula to generate the context vector $c^{<t_d>}$ in the attention module of the decoder at time t_d

Transformer:

$A = \text{softmax}\left(\frac{Q.K^T}{\sqrt{d_k}}\right)V$	self-attention module (without masking)
$A = \text{softmax}\left(\frac{Q.K^T}{\sqrt{d_k}} + M\right)V$	self-attention module (with mask)
$\tilde{X} = \frac{X - \text{mean}_{C,T}}{\text{std}_{C,T}}$	Layer normalization for input X of shape (B, T, C), B: batch size, T: sequence length, C: channel

Training NN:

$x_{std} = \frac{x-\mu}{\sigma}$	Standardization
$\hat{z}_c^{\{t\}} = \frac{z_c^{\{t\}} - \mu_c^{\{t\}}}{\sqrt{v_c^{\{t\}} + \epsilon}}$, $\tilde{z}_c^{\{t\}} = \gamma_c.\hat{z}_c^{\{t\}} + \beta_c$	Batch normalization (BN) during training for batch *t* and channel *c*.
$\mu_c^{\{t\}} = \frac{1}{B \times H \times W}\sum_{i=1}^{B}\sum_{h=1}^{H}\sum_{w=1}^{W} z_{c,h,w}^{\{t\}(i)}$	Mean of BN during training for an input of shape (B, H, W, C) B: batch size, H: height, W: height, C: channel size
$v_c^{\{t\}} = \frac{1}{B \times H \times W}\sum_{i=1}^{B}\sum_{h=1}^{H}\sum_{w=1}^{W}\left(z_{c,h,w}^{\{t\}(i)} - \mu_c^{\{t\}}\right)^2$	Variance of BN during training for an input of shape (B, H, W, C) B: batch size, H: height, W: height, C: channel size
$\vec{\mu}_c = (1-m).\vec{\mu}_c + m.\mu_c^{\{t\}}$	Running mean for BN during training
$\vec{v}_c = (1-m).\vec{v}_c + m.v_c^{\{t\}}.\frac{B \times H \times W}{B \times H \times W - 1}$	Running variance for BN during training
$J(W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]}) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}, y) + \lambda\frac{1}{2m}\sum_{l=1}^{L}\lVert W^{[l]}\rVert_2^2$	L2 regularization for an *L*-layered network, *m* samples
$J(W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]}) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}, y) + \lambda\frac{1}{2m}\sum_{l=1}^{L}\lvert W^{[l]}\rvert$	L1 regularization for an *L*-layered network, *m* samples
$var(W^{[l]}) = \frac{2}{(n^{[l-1]} + n^{[l]})}$	Xavier initialization
$var(W^{[l]}) = \frac{2}{n^{[l-1]}}$	Kaiming initialization